G I T

FIRST TIME

git init

git add .

git commit -m “Initial commit”

git branch -M main

git remote add origin https:github.com/your-username/project.git

git push -u origin main

If the folder lists are different in both places.

git pull origin main --allow-unrelated-histories

DAILY WORKFLOW

git status

git add .

git commit -m “Your message”

git push

What is Git

Git is a Version Control System.

Git helps you:

- Track code changes
- Save project history
- Work on projects safely
- Restore old code
- Collaborate with teams
- Upload projects to GitHub

What is GitHub

GitHub is a cloud platform where Git projects are stored online.

You can:

- Upload projects
- Share code
- Make portfolio
- Collaborate with others
- Contribute to open source
- Backup your code

BASIC GIT WORKFLOW

1. Create Project Folder
2. git init
3. Create files
4. git add .
5. git commit -m "message"
6. git remote add origin URL
7. git push -u origin main

---

1. CHECK GIT VERSION

Use

Check Git installed or not.

git --version

---

2. CONFIGURE GIT

Use

Set username and email.

Set Username

git config --global user.name "Your Name"

Set Email

git config --global user.email "youremail@gmail.com"

Check Config

git config --list

Check Username

git config user.name

Check Email

git config user.email

---

3. CREATE GIT REPOSITORY

Use

Initialize Git in project folder.

git init

---

4. CHECK STATUS

Use

See changed, staged, or untracked files.

git status

---

5. ADD FILES

Use

Move files to staging area.

Add Single File

git add filename

Add Multiple Files

git add file1 file2

Add All Files

git add .

---

6. COMMIT

Use

Save snapshot of project.

Commit

git commit -m "First Commit"

Commit Without Opening Editor

git commit -am "message"

Change Last Commit Message

git commit --amend -m "New Message"

---

7. VIEW COMMIT HISTORY

Use

See project history.

Show History

git log

One Line History

git log --oneline

Detailed History

git log --stat

Graph View

git log --oneline --graph --all

---

8. REMOVE FILES

Use

Delete tracked files.

Remove File

git rm filename

Remove Cached File Only

git rm --cached filename

Remove All .exe Files from Git

git rm --cached *.exe

---

9. RENAME FILES

Use

Rename files/folders.

git mv oldname newname

---

10. RESTORE FILES

Use

Undo changes.

Restore File

git restore filename

Restore All Files

git restore .

Unstage File

git restore --staged filename

---

11. DIFFERENCE CHECK

Use

See changes between versions.

Check Changes

git diff

Staged Changes

git diff --staged

---

12. BRANCHES

Use

Create separate working area.

Show Branches

git branch

Create Branch

git branch branch-name

Switch Branch

git checkout branch-name

Create + Switch Branch

git checkout -b branch-name

New Method

git switch branch-name

Create Using Switch

git switch -c branch-name

Rename Branch

git branch -m new-name

Delete Branch

git branch -d branch-name

Force Delete Branch

git branch -D branch-name

---

13. MERGE

Use

Combine branches.

Merge Branch

git merge branch-name

---

14. REMOTE REPOSITORY

Use

Connect local project to GitHub.

Add Remote

git remote add origin repository-url

Check Remote

git remote -v

Remove Remote

git remote remove origin

Change Remote URL

git remote set-url origin new-url

---

15. PUSH CODE TO GITHUB

Use

Upload code to GitHub.

First Push

git push -u origin main

Normal Push

git push

Push Specific Branch

git push origin branch-name

Force Push

git push --force

---

16. CLONE REPOSITORY

Use

Download GitHub repository.

git clone repository-url

---

17. PULL CODE

Use

Download latest changes.

git pull

Pull Specific Branch

git pull origin main

---

18. FETCH

Use

Get latest data without merging.

git fetch

---

19. CHECKOUT

Use

Switch commit or branch.

Switch Branch

git checkout branch-name

Checkout Commit

git checkout commit-id

---

20. RESET

Use

Undo commits.

Soft Reset

git reset --soft HEAD~1

Mixed Reset

git reset HEAD~1

Hard Reset

git reset --hard HEAD~1

---

21. REVERT

Use

Undo commit safely.

git revert commit-id

---

22. STASH

Use

Temporarily save changes.

Save Stash

git stash

Show Stash

git stash list

Apply Stash

git stash apply

Apply + Delete

git stash pop

Delete Stash

git stash drop

---

23. TAGS

Use

Mark versions/releases.

Create Tag

git tag v1.0

Show Tags

git tag

Push Tags

git push origin --tags

---

24. SHOW INFORMATION

Show Current Branch

git branch --show-current

Show Remote Info

git remote show origin

Show Commit Details

git show

---

25. .gitignore

Use

Ignore files/folders.

Create File

touch .gitignore

Example

node_modules/
.env
*.exe

---

26. GITHUB AUTHENTICATION

Login Using Browser

gh auth login

Check GitHub CLI

gh --version

---

27. GITHUB CLI COMMANDS

Create Repository

gh repo create

Clone Repository

gh repo clone repo-name

View Repository

gh repo view

Fork Repository

gh repo fork

---

28. FORK WORKFLOW

Clone Fork

git clone fork-url

Add Original Repository

git remote add upstream original-url

Fetch Original Repo

git fetch upstream

Merge Updates

git merge upstream/main

---

29. CHERRY PICK

Use

Copy specific commit.

git cherry-pick commit-id

---

30. REBASE

Use

Clean commit history.

Rebase Branch

git rebase main

Interactive Rebase

git rebase -i HEAD~3

---

31. CLEAN FILES

Use

Delete untracked files.

Show Files

git clean -n

Delete Files

git clean -f

Delete Files + Folders

```
git clean -fd
```

---

32. SEARCH COMMANDS

Search Word

```
git grep "word"
```

---

33. SHORTCUT COMMANDS

Short Status

```
git status -s
```

One Line Log

```
git log --oneline
```

---

34. SAVE CURRENT WORK QUICKLY

```
git add .
git commit -m "updated project"
git push
```

---

35. COMPLETE PROJECT UPLOAD

```
git init
git add .
git commit -m "First Commit"
git branch -M main
git remote add origin URL
git push -u origin main
```

---

36. CHANGE DEFAULT BRANCH TO MAIN

```
git branch -M main
```

---

37. DOWNLOAD SPECIFIC BRANCH

```
git clone -b branch-name repository-url
```

---

38. SEE WHO CHANGED LINE

```
git blame filename
```

---

39. SHOW FILE HISTORY

```
git log filename
```

---

40. SEE ALL CONTRIBUTORS

```
git shortlog -sn
```

---

41. COMPARE BRANCHES

git diff main branch-name

---

42. DELETE REMOTE BRANCH

git push origin --delete branch-name

---

43. PUSH ALL BRANCHES

git push --all origin

---

44. PULL REQUEST USING GITHUB CLI

gh pr create

---

45. OPEN REPOSITORY IN BROWSER

gh repo view --web

---

46. GITHUB PAGES DEPLOY

Install

npm install gh-pages --save-dev

Deploy

npm run deploy

---

47. SEE SAVED REMOTES

git remote

---

48. SHOW COMPLETE CONFIG

git config --global --list

---

49. REMOVE GIT FROM PROJECT

rm -rf .git

---

50. MOST IMPORTANT COMMANDS FOR BEGINNERS

git init
git status
git add .
git commit -m "message"
git push
git pull
git clone URL
git branch

git checkout branch-name
git merge branch-name

COMMON REAL LIFE EXAMPLE

Situation

You made a Student Management System project.

Steps

git init

git add .

git commit -m "Student Management System Added"

git branch -M main

git remote add origin URL

git push -u origin main

Now your project is uploaded to GitHub.

IMPORTANT NOTES

HEAD Meaning

HEAD

Means current commit position.

---

origin Meaning

origin

Means GitHub repository link.

---

main Meaning

main

Default branch name.

BEST PRACTICE

Good Commit Message

git commit -m "Added Login Page"

Bad Commit Message

git commit -m "update"

FINAL QUICK REVISION

Upload Project

```
git init
git add .
git commit -m "message"
git remote add origin URL
git push -u origin main
```

Update Project

```
git add .
git commit -m "updated"
git push
```

Download Project

```
git clone URL
```

Get Latest Changes

```
git pull
```

Create New Branch

```
git checkout -b feature-name
```